Machine Learning Experiment Guidebook

Experiment 3：CNN for MNIST Digit Recognition

1. Objective

The objective of this experiment is to explore the Convolutional Neural Network (CNN) model and its effectiveness in the field of handwritten digit recognition. By the end of this experiment, you should gain a fundamental understanding of CNN architectures, learn how to implement such a network in Python using PyTorch, and evaluate the model's performance in accurately classifying images of handwritten digits.

2. Environment

- Operating System: Windows 10
- Python programming environment: Anaconda & Jupyter Notebook

3. Experiment Tasks

3.1 Install Anaconda and Jupyter Notebook

Note: The lab computers have been installed with Anaconda.
You should have already installed Anaconda and Jupyter Notebook in Experiment 1. If not, please refer to the Experiment 1 guide for installation.

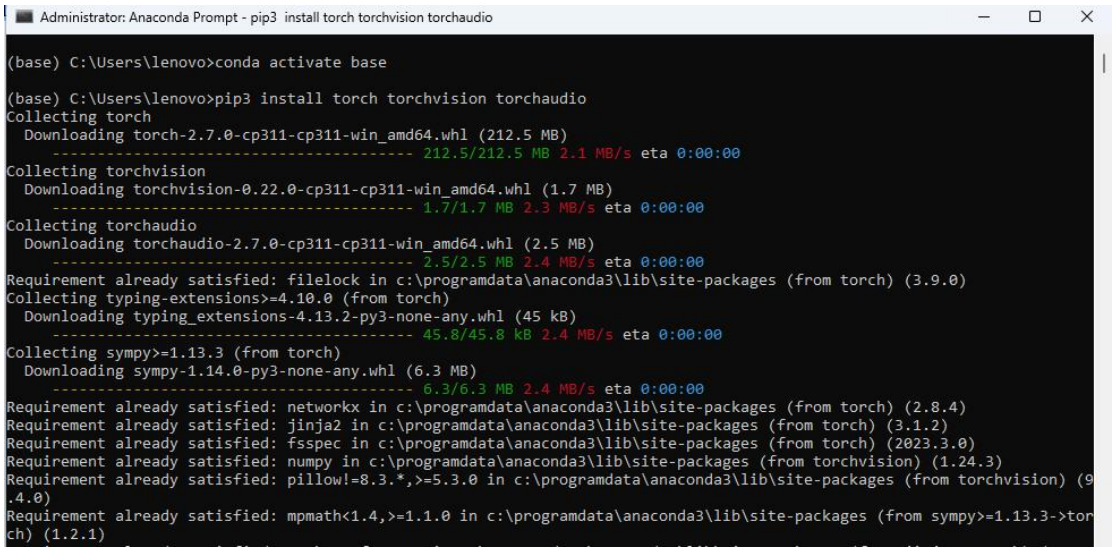
3.2 Set up PyTorch and DataLoader

(1) Install PyTorch and Torchvision

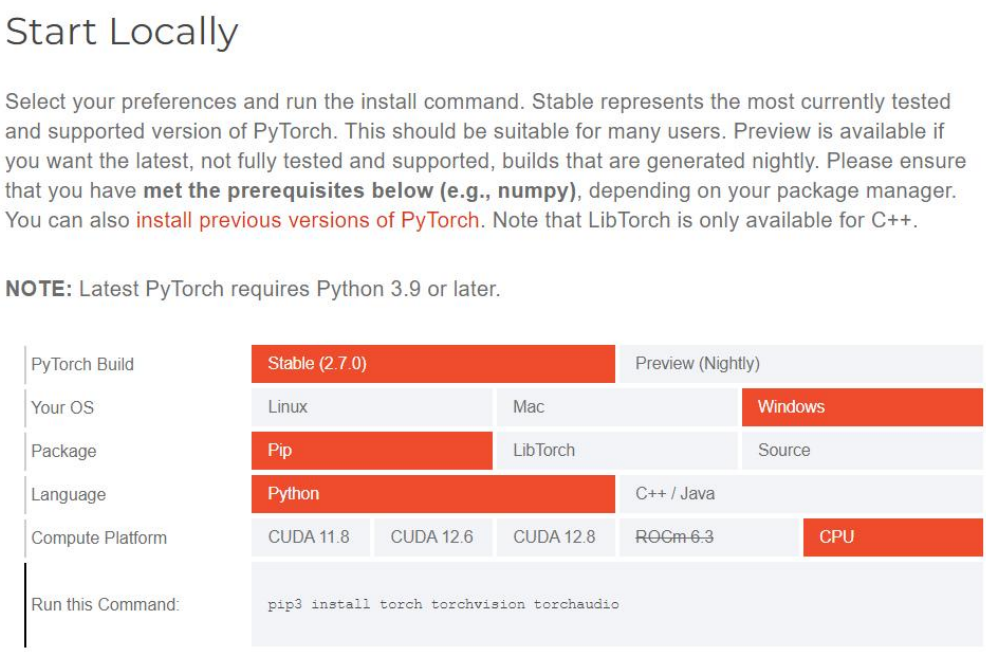
PyTorch is a deep learning framework that provides a wide array of tools and libraries for building deep learning models. Torchvision is an extension for PyTorch that provides popular datasets, model architectures, and common image transformations for computer vision.

To install PyTorch and Torchvision using pip, Python's package installer, you can run the following commands in Anaconda Prompt on the lab computers.

```
conda activate base
pip install torch torchvision torchaudio
```



On your own computer, you need to go to the PyTorch Get Started page (https://pytorch.org/get-started/locally/) to generate a custom install command. Pick your Python version, OS, and whether you want to use GPU (CUDA) support.



If you are not familiar with PyTorch, you can refer to the following tutorial.
Quickstart — PyTorch Tutorials 2.3.0+cu121 documentation

Note: Please follow the below-listed steps to to train a classifier using CNN algorithm in the MNIST dataset. Ensure to include screenshots of both the experimental code and its corresponding output results in your experiment report.

(2) Import required libraries

The following lines import the torch and torchvision libraries. These imports are necessary to utilize the functionalities provided by these libraries, such as datasets, model layers, and training utilities.

```
import torch
import torchvision
```

(3) Set up DataLoader

This line imports DataLoader from torch.utils.data. DataLoader is a versatile tool that provides the ability to load data from a dataset and offers various functionalities to manipulate the data before feeding it into a neural network. This is crucial for batching, shuffling, and loading the data in parallel.

```
from torch.utils.data import DataLoader
```

3.3 Configure Training Parameters

When developing a machine learning model, particularly in deep learning, setting the right training parameters is critical for model performance. These parameters, often referred to as hyperparameters, play a pivotal role in governing the training process and directly influence the effectiveness and efficiency of the learning algorithm.

```
n_epochs = 3
batch_size_train = 64
batch_size_test = 1000
learning_rate = 0.01
momentum = 0.5
log_interval = 10
random_seed = 1
torch.manual_seed(random_seed)
```

- Number of Epochs (n_epochs)

This specifies the total number of times the learning algorithm will work through the entire training dataset. Here, it's set to 3 epochs, meaning the entire dataset is passed forward and backward through the neural network three times.

- Batch Size

batch_size_train and batch_size_test define the number of training samples and test samples to work through before updating the internal model parameters. Smaller batch sizes often lead to better generalization of the model.

- Learning Rate

This is a hyperparameter that controls how much to change the model in response to the estimated error each time the model weights are updated. Setting this to 0.01 starts with a moderate rate of learning.

- Momentum

Momentum is a hyperparameter that helps the optimizer in accelerating gradients

vectors in the right directions, thus leading to faster converging. It is set to 0.5 in this case.

- Logging Interval (log_interval)

This parameter controls how often to log training progress. Here, progress will be logged every 10 batches of training.

- Random Seed

Setting a random seed ensures that the initial random weights of your model are the same every time you train. This is important for reproducibility of your results.

3.4 Load and Normalize the MNIST Dataset

The MNIST dataset is a large database of handwritten digits that is widely used for training and testing in the field of machine learning. It was created by Yann LeCun, Corinna Cortes, and Christopher Burges for evaluating machine learning models on the specific task of image recognition.

The dataset consists of 60,000 training images and 10,000 testing images, each representing a single digit from 0 to 9. Each image is a 28x28 pixel grayscale image, where each pixel represents an intensity from 0 to 255. This dataset is commonly used as a benchmark for evaluating the performance of various image processing systems and learning algorithms, particularly those in the field of deep learning.

Note: Please pay attention to indentation! Python uses it to define code blocks. Any code inside a class or function must be properly indented. You can use 4 spaces or a tab for each level, and please don't mix spaces and tabs, that will cause an error.

(1) Load MNIST Dataset

The torchvision.datasets.MNIST provides access to the MNIST dataset. DataLoader is a fundamental component in PyTorch, designed for efficient loading and batching of data during model training or evaluation. It helps in loading the data from the dataset, and provides functionalities such as batching, shuffling, and transforming the data.

✓ Initialize a DataLoader for training

If the download fails, you can try downloading again.

```
train_loader = torch.utils.data.DataLoader(
    torchvision.datasets.MNIST(
        './data/',
        train=True,
        download=True,
        transform=torchvision.transforms.Compose([
            torchvision.transforms.ToTensor(),
            torchvision.transforms.Normalize((0.1307,), (0.3081,))
        ])),
    batch_size=batch_size_train,
    shuffle=True
)
```

✓ train=True

It indicates that we're loading the training part of the dataset; if set to False, it would load the testing set.

✓ download=True

It means that if the MNIST dataset is not present in the specified path ('./data/' here), it will be downloaded from the internet automatically. If the dataset already exists in that path, it won't be downloaded again. The dataset required for this experiment has already been provided. You can place it in the same directory as the code files and use it directly.

✓ transform=torchvision.transforms.Compose([...])

Defines a sequence of transformations to be applied to the dataset images. It's a way to chain together multiple transforms. Here, it includes two transformations:

✓ ToTensor() converts the images into PyTorch tensors, which is required for computations within the neural network. In PyTorch, a Tensor is a fundamental concept representing a multi-dimensional array of numerical data, similar to NumPy's ndarray.

✓ Normalize() normalizes the tensor images with a mean of 0.1307 and a standard deviation of 0.3081, which are standard for MNIST.

✓ batch_size=batch_size_train

Determines the number of samples per batch that the DataLoader yields at a time.

✓ shuffle=True

When set to True, it shuffles the data indices before every epoch.

Note: If you encounter an error related to CERTIFICATE, you can try running the following commands.

```
pip install certifi
```

```
import os
import certifi
os.environ['SSL_CERT_FILE'] = certifi.where()
```

✓ Initialize a DataLoader for testing:

```
test_loader = torch.utils.data.DataLoader(
    torchvision.datasets.MNIST(
        './data',
        train=False,
        download=True,
        transform=torchvision.transforms.Compose([
            torchvision.transforms.ToTensor(),
            torchvision.transforms.Normalize((0.1307,), (0.3081,))
        ])),
    batch_size=batch_size_test,
    shuffle=True
)
```

(2) Enumerate and Access the Data

The enumerate() function allows us to loop over the dataset and returns both the

index and the data (images and labels).

```
examples = enumerate(test_loader)
batch_idx, (example_data, example_targets) = next(examples)
print(batch_idx)
print(example_data.shape)
print(example_targets)
```

These print statements show the labels of the digits in the first batch and the shape of the image data tensor. This means that we have 1000 examples of 28x28 pixel grayscale images (i.e., without RGB channels).

(3) Display Some Examples

```
# Import the matplotlib.pyplot module and alias it as 'plt'
import matplotlib.pyplot as plt
fig = plt.figure()

for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.tight_layout()
    plt.imshow(example_data[i][0], cmap='gray', interpolation='none')
    plt.title("Ground Truth: {}".format(example_targets[i]))
    plt.xticks([])
    plt.yticks([])
plt.show()
```

- ✓ plt.figure()：Create a new figure object, which will serve as the container for all subsequent subplots
- ✓ plt.subplot(2,3,i+1) creates a grid of 2 rows and 3 columns for the subplots.
- ✓ plt.tight_layout()：Apply tight layout settings to ensure proper spacing between subplots and labels
- ✓ plt.imshow() displays an image from the tensor data. The tensor slice '[i][0]' is used because the images are grayscale, and PyTorch stores them in the format '[batch index, color channel, height, width]'.
- ✓ plt.title() adds a title to each subplot indicating the actual digit.
- ✓ plt.xticks([])：Remove the x-axis tick marks and labels

Note: If you encounter a kernel crash when drawing graphs on the lab computer, you can try running the following commands.

```
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
```

3.5 Define the CNN Model

(1) What is CNN

Convolutional Neural Networks (CNNs) are a class of deep learning algorithms primarily designed for processing grid-like structured data, such as images, but also

applicable to other high-dimensional inputs like time-series data. Inspired by the organization and functioning of the visual cortex in animals, CNNs are highly effective at identifying and extracting features in visual data, making them instrumental in image recognition, computer vision, and related tasks.

Key components of a typical CNN include:

✧ Convolutional Layers

These layers apply a set of learnable filters (kernels) to the input data. Each filter slides across the entire input, performing element-wise multiplication and summing the result to create a feature map. This operation detects local patterns like edges or textures in the initial layers and more complex structures in deeper layers.

✧ Activation Functions

After each convolution, an activation function (commonly ReLU, or Rectified Linear Unit) is applied to introduce non-linearity into the model, enabling it to learn and represent complex relationships between features.

✧ Pooling Layers

Often following convolutional layers, pooling layers downsample the feature maps, reducing their spatial dimensions (width and height) while retaining important information. Max pooling, a common type, selects the maximum value from each sub-region of the feature map, thereby retaining salient features while decreasing computational complexity.

✧ Flatten layer

It converts the multidimensional output from the previous layers (typically convolutional and/or pooling layers) into a one-dimensional array or vector. This step is necessary before the data can be passed through fully connected layers, which require a one-dimensional input structure.

✧ Fully Connected Layers

Towards the end of the network, one or more fully connected layers (also known as dense layers) are used to combine the features extracted by previous layers for the final classification or regression output.

✧ Dropout

Techniques like dropout can be employed within these layers to prevent overfitting, where the model learns to memorize the training data instead of generalizing to unseen data.

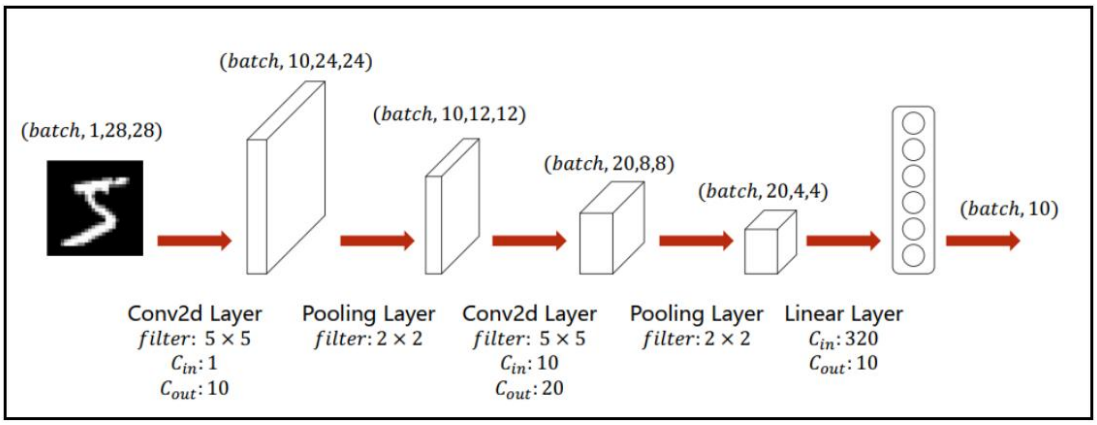
✧ Training and Backpropagation

CNNs are trained using gradient-based optimization algorithms, commonly stochastic gradient descent (SGD) or its variants, adjusting the weights of the kernels and fully

connected layers through backpropagation. The loss function, such as cross-entropy for classification tasks, measures the discrepancy between predicted and actual outputs and guides the learning process.

(2) CNN Architecture Definition

Now, let us define a simple CNN architecture suitable for classifying the MNIST digits.



The above is just a schematic diagram of the network structure. In the following example, we actually construct two fully connected layers. To build a model with pytorch, a class should be created. This class will contain an init with the different layers that will be used to define the architecture of the neural network. Then, the forward method will consist in building the network.

```
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

class Net(nn.Module):
    def __init__(self):
        super(Net,self).__init__()

        self.conv1 = nn.Conv2d(1, 10, kernel_size=5)
        self.conv2 = nn.Conv2d(10, 20, kernel_size=5)

        self.conv2_drop = nn.Dropout2d()

        self.fc1 = nn.Linear(320, 50)
        self.fc2 = nn.Linear(50, 10)

    def forward(self, x):
        x = F.relu(F.max_pool2d(self.conv1(x), 2))
        x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))
        x = x.view(-1, 320)
        x = F.relu(self.fc1(x))
        x = F.dropout(x, training=self.training)
        x = self.fc2(x)
        return F.log_softmax(x,dim=1)
```

✓ Define the structure of the neural network:

- class Net(nn.Module)

This defines a new class Net, inheriting from nn.Module, which is a base class for all neural network modules in PyTorch.

- self.conv1 and self.conv2 are 2D convolutional layers that help in extracting features from the input images.
- self.conv2_drop is a dropout layer applied to the second convolutional layer to prevent overfitting by randomly zeroing some of the elements of the input tensor.
- self.fc1 and self.fc2 are fully connected layers that map the learned features to the output classes.

✓ Define the forward pass of the network:

- x = F.relu(F.max_pool2d(self.conv1(x), 2))

Applies the first convolutional layer, followed by max pooling with a 2x2 window, and then a ReLU activation function.The second parameter "2" is the size of the pooling window, which is a 2x2 window. This means that for each 2x2 region in the input feature map, the function will select the maximum value among the four values as the output. At the same time, this will also reduce the height and width of the feature map by half.

- x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))

Applies the second convolutional layer, applies dropout, then max pooling with a 2x2 window, and ReLU activation.

- x = x.view(-1, 320)

Flattens the output from the convolutional layers to prepare it for the fully connected layer. -1 infers the correct number of rows based on the batch size, 320 = 20*4*4.

- x = F.relu(self.fc1(x))

Passes the data through the first fully connected layer with a ReLU activation.

- x = F.dropout(x, training=self.training)

Applies dropout to the output of the first fully connected layer. "self.training" is a boolean flag that is True during training and False during evaluation (testing).

- x = self.fc2(x)

Passes the data through the second fully connected layer.

- return F.log_softmax(x)

Applies the log softmax function to the final output, which is useful for the subsequent calculation of the loss during training.

(3) Network and Optimizer Initialization

Here, we use Stochastic Gradient Descent (SGD) as the optimization algorithm.

```
network = Net()
optimizer = optim.SGD(network.parameters(), lr=learning_rate, momentum=momentum)
```

- ✓ Net()：Instantiate the neural network model
- ✓ optim.SGD()：Initialize the optimizer for updating network parameters during training
- ✓ network.parameters() retrieves all trainable parameters within the network.
- ✓ lr stands for the learning rate, determining the step size at each iteration while moving toward a minimum of the loss function
- ✓ momentum is a parameter specific to SGD that helps accelerate the convergence by adding

a fraction of the previous update to the current one.

(4) Track Losses and Iteration Counters

Define the following lists to store losses and iteration counts for training and testing to monitor the model's performance across epochs.

```
# Initialize lists to track training and testing losses for visualization and analysis
train_losses = []
train_counter = []
test_losses = []
test_counter = []
test_counter = [i*len(train_loader.dataset) for i in range(n_epochs + 1)]
print(test_counter)
```

- ✓ train_losses：Records loss values for each batch during training
- ✓ train_counter：Stores the cumulative number of training samples processed
- ✓ test_losses：Records loss values for each epoch during testing
- ✓ test_counter：Prepares a list to plot test loss against the total number of training samples seen so far
- ✓ test_counter：It is calculated by multiplying the number of epochs (including the initial 0th epoch) by the total length of the training dataset, which provides a point of reference for each epoch end

3.6 Model Training and Evaluation

(1) CNN Training Function

This function manages the training process for a single epoch in a Convolutional Neural Network (CNN), iterating over batches of data from the train_loader, performing forward and backward passes, updating model parameters, and periodically logging progress and saving model states.

```
def CNNtrain(epoch):
    network.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        optimizer.zero_grad()
        output = network(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % log_interval == 0:
            print('Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}'.format(
                epoch, batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item()))
            train_losses.append(loss.item())
            train_counter.append((batch_idx*len(data)) + ((epoch-1)*len(train_loader.dataset)))
```

- Initialize Training Mode
  network.train() sets the module in training mode. It is important for layers with different behaviors during training vs. evaluation like “dropout”.
- Batch Processing

Iterates over train_loader, which yields batches of data and targets.

- Optimizer Zero Gradient
  optimizer.zero_grad() resets gradients to zero to avoid accumulation from previous iterations.
- Forward Pass
  network(data) forwards the batch through the network to get predictions.
- Calculate Loss
  F. nll_loss(output, target) computes the Negative Log-Likelihood loss between predictions and actual targets.
- Backward Pass
  loss.backward() computes the gradient of the loss with respect to all trainable parameters.
- Optimizer Step
  optimizer.step() updates the parameters based on gradients.
- Logging and Saving
  Every log_interval batches, prints the training progress and loss, appends loss to train_losses, tracks the number of examples seen in train_counter.

(2) CNN Testing Function

This function evaluates the performance of a trained neural network on a separate test dataset. It switches the network to evaluation mode, disables gradient calculations (which aren't needed during inference), computes the total loss and accuracy over the test set, and prints a summary of these metrics.

```
def CNNtest():
    network.eval()
    test_loss = 0
    correct = 0
    with torch.no_grad():
        for data, target in test_loader:
            output = network(data)
            test_loss += F.nll_loss(output, target, reduction='sum').item()
            pred = output.data.max(1, keepdim=True)[1]
            correct += pred.eq(target.view_as(pred)).sum()
    test_loss /= len(test_loader.dataset)
    test_losses.append(test_loss)
    print('\nTest set: Average loss: {:.4f}, Correct: {}/{} ({:.0f}%)\n'.format(
        test_loss, correct, len(test_loader.dataset),
        100. * correct / len(test_loader.dataset)))
```

- Set Evaluation Mode
  network.eval() sets the module in evaluation mode, affecting layers like dropout.
- Disable Gradient Computation
  with torch.no_grad() improves memory efficiency during inference.
- Accumulate Loss and Correct Predictions
  Loops through test_loader, computes loss without reducing, and counts correct predictions.
- Calculate Average Loss and Accuracy
  Computes the mean loss and accuracy percentage.
- Logging: Prints the average loss and accuracy for the test set.

(3) Train and Test Loop

Initiate the training and testing routine for a neural network model. It begins by conducting an initial test (CNNtest()) to measure the untrained model's performance, providing a benchmark against which improvements can be measured.

```
# Perform an initial test before training starts to establish a baseline performance
CNNtest()

# Loop through each epoch, training the network and then testing its performance
for epoch in range(1, n_epochs + 1):
    CNNtrain(epoch)
    CNNtest()
```

(4) Data Monitoring and Visualization

Uses matplotlib to plot training and testing losses to visualize the learning curve and model performance over epochs. It indicates how the loss decreases with each additional training example encountered. Test losses are marked as points on the graph, signifying the model's performance at the end of each epoch.

```
import matplotlib.pyplot as plt
fig = plt.figure()

plt.plot(train_counter, train_losses, color='blue')
plt.scatter(test_counter, test_losses, color='red')
plt.legend(['Train Loss', 'Test Loss'], loc='upper right')
plt.xlabel('number of training examples seen')
plt.ylabel('negative log likelihood loss')
plt.show()
```

- ✓ plt.plot()：Plot the training loss over the number of training examples seen
- ✓ plt.scatter()：Scatter plot for the test losses at each epoch end, highlighted in red

Observations: What do you discover from the results?

3.7 Visualize Predictions from a Trained CNN

(1) Model Prediction

Fetch a batch of test data from a data loader (test_loader), disabling gradient computation for efficiency during inference, and then passing this test data through a pre-defined neural network (network) to obtain the model's output predictions.

```
examples = enumerate(test_loader)
batch_idx, (example_data, example_targets) = next(examples)
with torch.no_grad():
    output = network(example_data)
```

(2) Visualization of Predictions

```
fig = plt.figure()
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.tight_layout()
    plt.imshow(example_data[i][0], cmap='gray', interpolation='none')
    plt.title("Prediction: {}".format(output.data.max(1, keepdim=True)[1][i].item()))
    plt.xticks([])
    plt.yticks([])
plt.show()
```

• Image Display: plt.imshow() displays an image from example_data. The data is accessed using indexing [i][0] to select the correct image and channel (grayscale images have one channel).

• Title with Prediction: The title for each image is set based on the model's prediction. output.data.max(1, keepdim=True)[1][i].item() extracts the predicted class from the model output.

Observations: What do you discover from the results?